Rapport d'inscription des étudiants

1 Introduction

Gérer des inscriptions revient à répéter quatre gestes : ajouter un étudiant, consulter la liste, corriger une fiche, supprimer une fiche. Le projet crud-minAppliInscriptionEtu les réalise en cinq fichiers PHP et une feuille de style, sans framework. Ce rapport suit l'application du point de vue de l'utilisateur, puis du code, et termine par ses limites.

2 Du geste de l'utilisateur à la requête SQL

Chaque action visible à l'écran correspond à un fichier, une méthode HTTP et une requête SQL :

Geste	Fichier	Méthode	Requête SQL
Afficher / rechercher	index.php	GET	SELECT … WHERE nom LIKE … OR prénom LIKE …
Ouvrir une fiche	form.php	GET	SELECT * FROM etudiant WHERE idEtu = ?
Créer	save.php	POST	INSERT INTO etudiant (…) VALUES (…)
Modifier	save.php	POST	UPDATE etudiant SET … WHERE idEtu = :idEtu
Supprimer	delete.php	POST	DELETE FROM etudiant WHERE idEtu = ?

Les lectures passent par GET, les écritures par POST. Ce choix a une conséquence de sécurité : ouvrir un lien ne peut jamais modifier les données.

3 Le code en détail

3.1 Un point d'entrée commun

Tous les fichiers commencent par charger config.php. Ce fichier ouvre la connexion PDO en mode exception, définit la fonction d'échappement h() et la constante CHAMPS, qui liste les douze colonnes de la table etudiant : idEtu, nomEtu, prenomEtu, dateNaiss, emailEtu, numTelEtu, regEtu, departEtu, arrondisEtu, mentionEtu, cycleEtu et ParcoursEtu. Cette liste unique évite de répéter les noms de champs ailleurs.

3.2 Lire et rechercher

La recherche utilise un paramètre nommé, ce qui garde la saisie séparée du SQL :

```
$stmt = $pdo->prepare(
  'SELECT idEtu, nomEtu, prenomEtu, emailEtu, mentionEtu, cycleEtu
     FROM etudiant
    WHERE nomEtu LIKE :q OR prenomEtu LIKE :q
    ORDER BY nomEtu');
$stmt->execute([':q' => "%$q%"]);
```

3.3 Un seul formulaire pour créer et modifier

form.php sert aux deux cas. Sans paramètre, les champs sont vides ; avec ?id=, la fiche est chargée et le matricule devient non modifiable, puisqu'il sert à identifier la ligne. Les champs sont regroupés en quatre blocs (Identité, Contact, Localisation, Formation) uniquement pour la lisibilité. Un champ caché indique à save.php s'il s'agit d'une création ou d'une mise à jour.

3.4 Enregistrer sans répéter le code

save.php vérifie d'abord chaque champ et l'adresse e-mail, car le navigateur peut être contourné. Il

construit ensuite la requête à partir de CHAMPS :

```
$set = implode(', ', array_map(fn($c) => "$c = :$c"
 array_diff(CHAMPS, ['idEtu'])));
$pdo->prepare("UPDATE etudiant SET $set WHERE idEtu = :idEtu")
->execute($d);
```

Les noms de colonnes viennent d'une constante et non de la saisie, donc cette construction dynamique reste sûre ; seules les valeurs transitent par des paramètres. En cas d'échec, le code d'erreur 23000 (doublon) est converti en message clair : « Matricule ou email déjà utilisé ».

3.5 Supprimer

Chaque ligne du tableau contient son propre petit formulaire POST, précédé d'une confirmation JavaScript. delete.php répond par une erreur 405 à toute autre méthode.

4 Bilan de sécurité

L'application protège bien contre l'injection SQL (requêtes préparées partout) et contre le XSS (h() sur chaque affichage, y compris le message d'erreur transmis dans l'URL). Elle ne protège pas encore contre les requêtes forgées : aucun jeton CSRF n'accompagne les formulaires. La connexion utilise le compte root sans mot de passe, ce qui est normal sous XAMPP en local mais inacceptable sur un serveur réel.

5 Limites et corrections proposées

> **À corriger en priorité.** le commentaire de config.php cite la base « universiteMbdTest » alors que le code se connecte à « universite ». Il faut créer la base sous le nom que le code utilise.

- ✧ **Pas de script SQL fourni :** la base et la table etudiant doivent exister avant le premier lancement.
- ✧ **Suppression trop optimiste :** delete.php ne vérifie pas l'existence du matricule et affiche toujours « Opération réussie ».
- ✧ **Recherche :** les caractères % et _ saisis ne sont pas échappés, et le LIKE %…% ne profite pas d'un index.
- ✧ **Validation légère :** seuls l'e-mail et la présence des champs sont contrôlés ; téléphone et date de naissance pourraient l'être.
- ✧ **Pas de pagination :** acceptable pour un TP, à ajouter avec LIMIT/OFFSET sur de gros volumes.

6 Conclusion

Pour un TP, le projet est réussi : le code est court, cohérent et sûr sur l'essentiel. Il peut servir de base à une version plus solide en ajoutant un jeton CSRF, un compte MySQL dédié, un script de création de la base et un contrôle d'existence à la suppression.

Mise en route : démarrer Apache et MySQL dans XAMPP, créer la base universite avec sa table etudiant, copier le dossier dans C:\xampp\htdocs\, puis ouvrir http://localhost/crud-minAppliInscriptionEtu/.